



Bil495 - Innovative Computer Application Lord of the Strings

Submitted By

Aylin Doğan 221101058

Emir Yücedağ 221101077

Kerem Kazandır 221101007

Zeynep Yağmur Bozdağ 221101078

Department of Computer Engineering
TOBB University of Economics & Technology

Spring 2025

ABSTRACT

A brief description of the project.

TABLE OF CONTENTS

ABSTRACT	2
TABLE OF CONTENTS	2
1. INTRODUCTION	1
1.1. Project Purpose	1
1.2. Project Scope	1
1.3. Definitions, Acronyms, and Abbreviations	1
1.4. References	1
1.5. Overview	1
2. Overall Description	2
2.1. Product Perspective	2
2.2. Product Functions	2
2.3. User Characteristics	2
2.4. Constraints	2
2.5. Assumptions and Dependencies	2
3. Specific Requirements	3
3.1. External Interface Requirements	3
3.1.1. User Interfaces	3
3.1.2. Hardware Interfaces	3
3.1.3. Software Interfaces	3
3.1.4. Communication Interfaces	3
3.2. Functional Requirements	3
3.3. Performance Requirements	3
3.4. Design Constraints	3
3.5. Software System Attributes	3
3.5.1. Reliability	3
3.5.2. Availability	3
3.5.3. Security	3
3.5.4. Maintainability	3
3.5.5. Portability	3
3.6. Other Requirements	3
4. DATASET	4
4.1. DATASET DESCRIPTION	4
4.2. DATA COLLECTION (IF APPLICABLE)	4
4.3. Data Storage	4
4.4. DATA PREPROCESSING	4
4.5. DATA LIMITATIONS AND ASSUMPTIONS (IF APPLICABLE)	4

5. System Architecture and Technical Design	4
5.1. Architectural Design	5
5.2. Technology Stack	5
5.3. IMPLEMENTATION DETAILS	5
6. Testing & Quality Assurance	6
6.1. Software Quality Assurance Plan (SQAP)	6
6.2. Verification and Validation (V&V)	6
6.3. Configuration and Change Management	6
6.4. Risk and Defect Management	6
6.5. Product Evaluation and Acceptance	6
7. Project Management and Risk Analysis	7
7.1. Project Plan	7
Gantt Chart for 12 Weeks	7
7.2. Task Distribution	7
7.3. Risk Analysis and Mitigation Strategies	7
8. Use Cases and User Stories	8
8.1. Target Audience	8
Who faces the problem most?	8
8.2. User Stories	8
9. Social, Environmental, and Legal Impact	9
9.1. SOCIETAL BENEFITS	9
9.2. ECONOMIC CONSTRAINTS	9
9.3. LEGAL AND ETHICAL COMPLIANCE	9
REFERENCES	9
APPENDIX	10

1. INTRODUCTION

1.1. Project Purpose

In today's fast-paced world, people struggle to manage their free time effectively. Between work, classes, and daily responsibilities, individuals who want to engage in social activities often face uncertainty about who to meet, where, and when. This leads to many wasted opportunities for social interaction and personal development, as people either spend their free time unproductively or fail to connect with others who share similar interests.

Existing social media and event platforms primarily focus on broad networking or large-scale events, but they do not take into account users' real-time availability. As a result, they are ineffective in facilitating spontaneous meetups or personalized activity planning. Even when people try to plan events in advance, differences in schedules and limited visibility into others' availability often make coordination difficult. This lack of synchronization discourages people from initiating activities, ultimately reducing opportunities for social engagement, collaboration, and building meaningful connections.

A solution is needed that can dynamically match individuals based on their free time and preferences, enabling them to discover, create, and join activities that fit naturally into their daily lives.

1.2. Project Scope

Our product will make an impact as a brand new social media type. We will increase the competitiveness and market growth in this field by making unknown brands in the entertainment industry more accessible through our application.

On the other hand our app has no concern about replacing other social media platforms. It will have its own uniqueness.

For the larger scale our app can have some legal issues and could have some security problems because of the physical meetings. So we are making it for students because It's easier to authenticate via their emails.

1.3. Definitions, Acronyms, and Abbreviations

UI: User Interface

UX: User Experience

API: Application Programming Interface

GDPR: General Data Protection Regulation

KVKK: Kişisel Verileri Koruma Kanunu

1.4. References

We will update this part when related works are used.

1.5. Overview

2. part is about general description you can find general summarization of the app.
3. part is about more detailed descriptions and requirements which app have.
4. part is about data storage and dataset which will be used for recommendation.
5. part is on which systems, libraries and frameworks we use.
6. part is about which test methods we use, their impact on quality, and which systems we use to ensure quality.
7. part contains information about the expected construction process of the project and indicates who will be responsible for which part.
8. part helps the user to briefly understand what to expect from the application.
9. part addresses the social, economic and legal effects and restrictions of the app.

2. OVERALL DESCRIPTION

2.1. Product Perspective

Explain how the software relates to existing systems and what dependencies it has.

2.2. Product Functions

Provide a high-level summary of system capabilities.

2.3. User Characteristics

Define the target audience and their level of expertise.

2.4. Constraints

List any technical, legal, or operational limitations.

2.5. Assumptions and Dependencies

Identify factors that affect the system but are outside of direct control.

3. SPECIFIC REQUIREMENTS

This is the most crucial part of the report, detailing every functional and non-functional requirement.

3.1. External Interface Requirements

3.1.1. User Interfaces

UI/UX considerations, dashboard, API

3.1.2. Hardware Interfaces

Cloud infrastructure, GPUs, IoT devices

3.1.3. Software Interfaces

Integrations with existing platforms

3.1.4. Communication Interfaces

Data pipelines, streaming protocols

3.2. Functional Requirements

- Define the software features, how they work, and their expected behavior.
- Each requirement should be in “The system shall...” format.

3.3. Performance Requirements

Data processing speed, model training time, latency, and real-time requirements.

3.4. Design Constraints

Compliance with regulations (GDPR, HIPAA, etc.), software limitations, and scalability.

3.5. Software System Attributes

3.5.1. Reliability

System uptime, failover mechanisms

3.5.2. Availability

Expected operational hours, redundancy

3.5.3. Security

Encryption, user authentication

3.5.4. Maintainability

Code modularity, logging

3.5.5. Portability

Cross-platform considerations

3.6. Other Requirements

Any additional project-specific criteria.

4. DATASET

This section provides details about the dataset(s) used in the project, including their sources, characteristics, and relevance to the problem being addressed. If the project involves collecting, cleaning, or preprocessing data, describe those processes here.

4.1. Dataset Description

We did not receive any data yet. We are looking for suitable datasets. Furthermore we will make a special dataset for this app with existing datasets and our producing data.

4.2. Data Collection (If Applicable)

4.3. Data Storage

At this stage, we are currently considering a MySQL and JavaScript based solution, as the data size of the project is not very high.

4.4. Data Preprocessing

We are planning to create a customized dataset using the parts of the datasets we find online that are useful to us and the data we produce ourselves. After the data is produced, we consider applying the necessary pre-processing steps depending on the situation.

4.5. Data Limitations and Assumptions (If Applicable)

5. SYSTEM ARCHITECTURE AND TECHNICAL DESIGN

This section details the methods and processes used to develop the project. It covers requirement gathering, system design, implementation strategies, and the selection of tools and technologies.

5.1. Architectural Design

The Gather Up platform utilizes a **Modular Monolith Architecture** hosted within a cloud-based **Application Server (Node.js/Express.js Runtime)**. This structure balances the speed of a single deployment unit with the organizational benefits of microservices, aligning well with the rapid iteration needs of a university-based social platform. The system is layered into three primary tiers: Presentation, Application, and Persistence.

5.1.1. Final System Architecture Overview

The system is visualized as a unified structure emphasizing the data flow between core components.

- **Presentation Layer:** Handled by the **Mobile Client (React Native)**.
- **Application Layer:** Contains the **Application Server** hosting the business logic (Matching Engine, Event, and User Management Modules).
- **Persistence Layer:** Managed by the **MySQL Database**.

5.1.2. Component Deployment and Interaction (Deployment View)

The system's physical deployment involves four key nodes interacting over secure channels:

- **Application Server** $\rightarrow$ **Database Server (MySQL)**: Communicates via **TCP/IP (secure DB connection)** for persistent data operations.
- **Application Server** $\rightarrow$ **Mobile Client**: Communicates via **HTTPS (API calls)** for general operations and **WebSocket** for real-time chat functionality.
- **Application Server** $\rightarrow$ **FCM Service**: Communicates via **HTTPS** to send push notifications to users.

5.1.3. Internal Modular Design (Application Layer Detail)

The **Application Server** is structured as a **Modular Monolith**, where key functionalities are encapsulated as independent modules that communicate internally:

- **Authentication Module:** Acts as the gateway, validating all incoming requests.
- **Matching Engine (Core Logic):** The central algorithm, receiving **availability data** from the **Real-Time Availability Module** and **preferences/feedback** from the **User Management Module** to generate event recommendations.
- **Chat Module (WebSocket Server):** Handles real-time communication between accepted event participants.
- **Event Management Module:** Controls the lifecycle and state of all events (Created, Open, Active, Completed, Archived).

5.2. Technology Stack

The following frameworks, libraries, and tools were selected based on their performance, scalability, and ease of development:

Category	Technology	Purpose
Backend Runtime	Node.js	Execution of server-side code.
Backend Framework	Express.js	Creating RESTful API endpoints and managing server logic.
Mobile Development	React Native	Developing the mobile application for iOS and Android using a single codebase.
Database	MySQL	Persistent and secure storage of relational data.
Real-Time Communication	WebSockets	Low-latency, bi-directional communication for the chat module.
Notifications	Firebase Cloud Messaging (FCM)	External, reliable push notification service.
AI/ML Logic	(If Necessary) Python Libraries (NumPy, SciPy)	For complex similarity score calculations within the Matching Engine.

5.3. Implementation Details

5.3.1. Programming Logic: The Matching Engine

The core value of the platform relies on the **Matching Engine** logic. This logic is implemented as a scoring algorithm that combines three primary factors:

1. **Availability Overlap:** Binary score (1 or 0) indicating if the event's time falls within the user's available time slot.
2. **Interest Similarity Score:** Calculated using a weighted average or vector similarity based on user and event tags/preferences.
3. **Proximity/Location Weight:** A decay function applied based on the distance between the user's current location and the event location, prioritizing closer events.

$$\text{Total Score} = (\text{Availability} \times 10) + (\text{Interest Score} \times W_i) + (\text{Proximity Weight} \times W_p)$$

The final scores are used to rank and present the Top N event recommendations to the user.

5.3.2. State Management Implementation

The system rigorously enforces lifecycle management using State Machine logic to ensure data integrity, especially for events and user relationships:

- **Event Lifecycle State Machine:** Manages transitions from **Created** $\rightarrow$ **Open/Ready** $\rightarrow$ **Active** $\rightarrow$ **Completed** $\rightarrow$

Archived. The **Completed** state triggers the mandatory **Safety Score update** process.

- **User Relationship State Machine:** Manages friendship/interaction states (**Pending, Accepted, Blocked**).

5.3.3. Security and Ethical Implementation

A key focus is placed on the ethical and safety aspects:

- **Authentication:** All user interaction relies on secure, token-based authentication (JWT/OAuth) managed by the **Authentication Module**.
- **Safety Score:** The **User Management & Feedback Module** implements the **Safety Score** as a dynamic reliability rating based on post-event feedback (ratings, attendance confirmation). This score influences the matching priority and acts as a preventative measure against misuse.

6. TESTING & QUALITY ASSURANCE

This section defines how the software reliability and quality of the Gather Up platform will be ensured throughout the project, with a core focus on real-time matching algorithm performance and user data security.

6.1. Software Quality Assurance Plan (SQAP)

The primary goal of the Quality Assurance Plan is to verify that the software meets both functional requirements (specifically matching accuracy and event creation) and non-functional requirements (real-time performance and data security). Our approach adopts an Agile development methodology, integrating Quality Assurance (QA) into continuous, iterative cycles. Functional and regression testing will be conducted at the end of every sprint and supported by continuous integration (CI) processes. For establishing relevant standards, the ISO/IEC 25010 (SQuaRE) standard will be utilized, focusing on the sub-characteristics of Usability, Performance Efficiency, and Security. Additionally, fundamental clean code principles and established secure coding best practices will serve as the primary guidelines for maintaining code quality and integrity.

6.2. Verification and Validation (V&V)

A multi-layered testing approach will be implemented to ensure the platform correctly implements all requirements and meets user needs.

- **Testing Approach:** Testing will progress through four distinct levels. Unit Tests will verify the correctness of the smallest code units, including algorithm logic, data processing functions, and critical business rules (like availability checks). Integration Tests will focus on data flow between the matching algorithm and the database, and the secure integration with third-party services (e.g., mapping and geolocation Open APIs). System Tests will evaluate the entire system as a cohesive whole, including performance tests to measure the real-time matching speed and security tests to confirm authorization and data protection mechanisms. Finally, Acceptance Tests (UAT) will involve the target users (university students, young professionals) to validate the application's ease of use and adherence to functional goals.
- **Coverage Metrics:** Quality will be evaluated through quantitative metrics. Requirement-Based Testing will track the completion percentage of test scenarios corresponding to each core objective (smart matching, event creation, feedback system). Fault Density (the number of defects found per module, such as the matching module) will be utilized to gauge the underlying quality of the code base.
- **Security and Compliance:** Security is a critical V&V area. Compliance with GDPR and local data protection laws will be ensured through regular reviews of data processing flows and user consent mechanisms. Furthermore, adherence to secure coding practices is mandatory to mitigate common vulnerabilities identified in frameworks like the OWASP Top 10.

6.3. Configuration and Change Management

- **Version Control:** The entire codebase and project assets will be managed using Git. Transitions between development, feature, and main branches will be strictly governed by a controlled Git workflow (e.g., Gitflow or GitHub Flow).

- **Change Tracking and CI/CD:** All change requests and reported issues will be meticulously tracked using a ticket/issue tracking system (e.g., GitHub Issues). No changes will be merged into the main codebase without passing through Continuous Integration/Continuous Deployment (CI/CD) pipelines that automatically execute all necessary tests. Mandatory peer code reviews will be enforced for every code integration to ensure quality and adherence to standards.

6.4. Risk and Defect Management

Software defects will be systematically recorded, managed, and resolved using the designated issue tracking system.

- **Tracking and Classification:** Every identified defect will be classified based on its severity (Critical, High, Medium, Low) and priority. Critical (e.g., data breach or a fundamental matching function crash) and High priority defects must be addressed and resolved before the start of the next development sprint.
- **Resolution and Verification:** Once a defect is fixed by the development team, it must be thoroughly verified and approved by the QA team before the issue can be formally closed, ensuring the fix did not introduce new regressions.

6.5. Product Evaluation and Acceptance

The final acceptance of the software is contingent upon proving the successful achievement of the MVP goals and meeting defined critical performance thresholds.

- **Acceptance Criteria:** Acceptance requires the platform to meet specific criteria across multiple dimensions: Functionality (all core features—availability input, smart matching, event creation, participation, and feedback—must be 100% operational according to requirements); Performance (the real-time matching algorithm must return results within a defined maximum response time, such as under 2 seconds); Security (successful security and penetration test results with proven data privacy); and Usability (the user interface must pass a predefined satisfaction threshold established during UAT).
- **Error Handling:** Acceptance is contingent upon zero open defects classified as Critical or High severity. The system's robustness in handling expected errors and exceptions must be documented and verified.
- **Review Processes:** To ensure software readiness, peer reviews are mandatory during the development phase. Furthermore, a final audit will be conducted at the end of the project to verify compliance with all legal, ethical, and lifecycle requirements, providing final validation of the product's quality and fitness for deployment.

7. PROJECT MANAGEMENT AND RISK ANALYSIS

7.1. Project Plan

Gantt Chart for 12 Weeks.



7.2. Task Distribution

Describe how the workload was divided among team members. Include a table or bullet points specifying which team member worked on each part of the project. If applicable, mention any pair programming or collaborative tasks. You can add other rows if exists.

Task	Team Member(s)	Description
UI/UX Design	Aylin, Yağmur	Created wireframes and designed the user interface.
Backend Development	Emir, Kerem	Implemented the database and API logic.
Frontend Development	Aylin, Yağmur	Developed the client-side functionality.
Testing & Debugging	Yağmur	Conducted unit tests and fixed reported bugs.
AI Development	Kerem	Developed Recommendation System for Events.
Documentation & Report Writing	All Team	Compiled the final project report.

7.3. Risk Analysis and Mitigation Strategies

We are aiming to make our own dataset so while making it we will try to lower bias. Because of we did not choose which models will be used, we do not have any idea of explainability.

Our model will use data from users and events with that we will modify our model. And we will implement all of this in accordance with KVKK and GDPR.

8. USE CASES AND USER STORIES

8.1. Target Audience

Students and young professionals who want to quickly find out what activities they can participate in during their free time.

8.2. User Stories

As a student, I want to organize an event with my friends.

As a parent, I want to know if my child is using a secure app.

As a young professional, I want to participate in an event after my job.

As a shop owner, I want more people to come into my shop and use it.

As a student club president, I want more people to be aware of the activities we do as a club.

As a university administrator, I want to track the number of active student events so that I can allocate resources effectively.

As a new student, I want to discover nearby social or academic events so that I can make new connections.

As an event organizer, I want to collect feedback from participants so that I can improve future events.

9. SOCIAL, ENVIRONMENTAL, AND LEGAL IMPACT

9.1. Societal Benefits

Our event matching system will help to manage time management. With that people will have a more organised life.

9.2. Economic Constraints

We will use design patterns for coding with that we are aiming to increase code efficiency and performance efficiency

For the app we will use a cloud provider (like Google Cloud or Azure) that is publicly committed to 100% renewable energy for their data centers..

9.3. Legal and Ethical Compliance

We will use the data of our users and activities according to GDPR and KVKK standards.

REFERENCES

List all the sources cited throughout the report using the **IEEE citation format**. This section should include books, research papers, online articles, datasets, and any other references used for literature review, methodology, or implementation.

What to Include in This Section:

- Follow **IEEE citation style**, which uses **numerical referencing** (e.g., [1], [2]).
- Ensure all references are **properly formatted** with full details (authors, title, publication, year, and DOI/link if applicable).
- Cite **academic and reliable sources** instead of unverified websites.

Example reference (IEEE format):

[1] A. Smith, “Introduction to Machine Learning,” Journal of AI Research, vol. 34, no. 2, pp. 101-120, 2023.

APPENDIX

The appendix contains **supplementary materials** that are relevant to the project but not included in the main report body. It provides additional technical details, large datasets, code snippets, or extended explanations that support the content discussed in earlier sections.

What to Include in This Section:

- **Extended tables, figures, or raw data** that are too detailed for the main text.
- **Source code snippets or algorithms** (if necessary) to clarify implementation details.
- **Detailed mathematical derivations or calculations** used in the project.
- **Additional documentation or survey results** (if user testing or external data collection was conducted).

If multiple appendices are included, label them as **Appendix A, Appendix B, etc.** and refer to them in the main report as needed.